React · folha de ajuda

A frente tem o que precisas para escrever código. O verso tem o que fazer quando algo dá erro.

No terminal dentro da pasta `client`

`npm install`	Instala o que está no `package.json`. Uma vez, depois de copiares ou clonares o projeto.
`npm run dev`	Arranca a app em `http://localhost:5173`. Deixa este terminal aberto.
`Ctrl + C`	Para a app.
`cd client`	Entra na pasta da app. O `npm` só funciona na pasta que tem o `package.json`.
`F12`	No browser: abre a **consola**, onde aparecem os avisos do React, e o **Network tab**.

Não é preciso reiniciar nada quando guardas um ficheiro: o browser atualiza sozinho.

Um componente, por partes

```jsx
// Card.jsx
import { useState } from "react"; 1

function Card({ name, attack }) { 2
  const [likes, setLikes] = useState(0); 3

  return (
    <li className="card"> 4
      <b>{name}</b> — ataque {attack} 5
      {attack >= 6 && <span>forte</span>}
      <button onClick={() => setLikes(likes + 1)}> 6
        ♥ {likes}
      </button>
    </li>
  );
}

export default Card; 7
```

1. Só é preciso importar o `useState` se o componente tiver estado.
2. Nome com **maiúscula**. As props chegam entre `{ }`.
3. **Estado:** o valor atual e a função que o muda.
4. O `return` devolve **um** elemento. `className`, não `class`.
5. Tudo o que é JavaScript vai entre `{ }`.
6. Nos eventos passas **uma função**: `() => …`
7. Sem `export default`, o ficheiro não pode ser importado.

As regras do JSX

`className` Em vez de `class`.	**Um elemento só** Dois lado a lado vão dentro de `<> … </>`.	**Componentes com maiúscula** `<Card />` sim, `<card />` não.	**`{ }` para JavaScript** `{cards.length}`, `{card.name}`.
Todas as tags fecham `<input />`, `<img />`, ` `.	**Eventos em camelCase** `onClick`, `onChange`, `onSubmit`.	**`key` em cada `.map()`** Um valor diferente para cada item.	**Comentários** `{/* assim */}` dentro do JSX.

Padrões que vais usar

Mostrar uma lista

```jsx
{cards.map((card) => (
  <Card key={card.name} {...card} />
))}
```

Mostrar só se...

```jsx
{attack >= 6 && <span>forte</span>}
```

Isto ou aquilo

```jsx
{cards.length === 0
  ? <p>Sem cartas.</p>
  : <ul>…</ul>}
```

Um botão que muda o estado

```jsx
const [filter, setFilter] = useState("Todas");

<button onClick={() => setFilter("Criatura")}>
  Só criaturas
</button>
```

Uma caixa de texto

```jsx
const [search, setSearch] = useState("");

<input
  value={search}
  onChange={(e) => setSearch(e.target.value)}
/>
```

Filtrar antes de mostrar

```jsx
const visible = cards.filter(
  (card) => filter === "Todas" || card.type === filter
);
// depois: visible.map(...)
```

Quando der erro

Procura aqui a mensagem que estás a ver. Os erros de escrita aparecem no ecrã do browser e no terminal; os avisos aparecem na consola (F12).

A MENSAGEM	O QUE QUER DIZER	COMO RESOLVER
`Adjacent JSX elements must be wrapped in an enclosing tag.` ECRÃ	O `return` tem dois elementos lado a lado.	Põe tudo dentro de `<> … </>` ou de uma `<div>`.
`Expected corresponding JSX closing tag for 'input'.` ECRÃ	Há uma tag por fechar. Em JSX, todas fecham.	`<input />`, `<img />`, ` `. Se for outra tag, procura o `</…>` que falta.
`Failed to resolve import "./Card.jsx" from "src/App.jsx". Does the file exist?` ECRÃ	O ficheiro do `import` não existe com esse nome.	Confirma o nome, a pasta e as maiúsculas: `Card.jsx` e `card.jsx` são ficheiros diferentes.
`Too many re-renders. React limits the number of renders to prevent an infinite loop.` ECRÃ · CONSOLA	Chamaste a função logo, em vez de a passares ao evento.	`onClick={setFilter("x")}` → `onClick={() => setFilter("x")}`
`cards is not defined` CONSOLA	Estás a usar um nome que não existe nesse ficheiro.	Falta um `import`, a variável tem outro nome, ou falta passá-la como prop.
`Each child in a list should have a unique "key" prop.` CONSOLA	Um `.map()` sem `key`.	`<Card key={card.name} … />`
`Invalid DOM property `class`. Did you mean `className`?` CONSOLA	Usaste `class`.	Troca por `className`.
`The tag <card> is unrecognized in this browser.` CONSOLA	O componente está escrito em minúsculas, e o React julga que é uma tag de HTML.	Maiúscula no nome da função, no `import` e na tag: `<Card />`.
A página fica em branco.	Há um erro algures, mas não aparece no ecrã.	Abre a consola (F12): o erro está lá. Confirma também o `export default`.
Carrego no botão, a variável muda, mas a página não.	É uma variável normal, e o React não sabe que ela mudou.	Usa `useState` e muda o valor com a função `set…`.
`'vite' is not recognized as an internal or external command` *ou «não é reconhecido como um comando interno»* TERMINAL	O `npm install` ainda não correu nesta pasta.	`cd client`, `npm install`, e depois `npm run dev`.
`npm error enoent Could not read package.json` TERMINAL	Estás na pasta errada.	`cd client`. A pasta certa é a que tem o `package.json`.
`Port 5173 is in use, trying another one…` TERMINAL	Já tens outra app aberta nessa porta.	Abre o endereço que o terminal mostrar (por exemplo, 5174), ou fecha a outra com `Ctrl + C`.

Antes de chamares o professor

1. **Lê a mensagem em voz alta.** Ela diz o ficheiro e a linha.
2. **Abre a consola** (F12). Muitos erros só aparecem lá.
3. **Guardaste o ficheiro?** O terminal do `npm run dev` ainda está a correr?
4. **Três minutos sozinho**, depois o teu par, só depois o professor.

Documentação oficial, em português: **pt-br.react.dev/learn**

Podes usar IA para aprender. Mas tens de conseguir explicar e alterar qualquer linha que entregas. Se não conseguires, essa parte vale zero.

Bons usos

- «Explica-me esta mensagem de erro.»
- «O que faz esta linha?»
- «Dá-me um exemplo de `useState` com outro tema.»

O que não conta como teu

- Colar o enunciado e entregar a resposta.
- Código com coisas que nunca viste e não sabes explicar.

No README, acrescenta uma linha: «Usei IA para …». Ser honesto sobre isso não tira nota.